

train_test_split#

[https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.](https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html)

Description:

Split arrays or matrices into random train and test subsets. Quick utility that wraps input validation, `next(ShuffleSplit()).split(X, y)`, and application to input data into a single call for splitting (and optionally subsampling) data into a one-liner. Read more in the User Guide. Allowed inputs are lists, numpy arrays, scipy-sparse matrices or pandas dataframes.

Function Signature

```
sklearn.model_selection.train_test_split(*arrays,  
test_size=None, train_size=None, random_state=None,  
shuffle=True, stratify=None)[source]#
```

Parameters:

**arrays*sequence of indexables with same length / shape[0]

*test_size*float or int, default=None

*train_size*float or int, default=None

```
sklearn.model_selection.train_test_split(*arrays,  
test_size=None, train_size=None, random_state=None,  
shuffle=True, stratify=None)[source]#
```

Parameters:

**arrays*sequence of indexables with same length / shape[0]

*test_size*float or int, default=None

*train_size*float or int, default=None

Parameters

stratifyarray

If not None, data is split in a stratified fashion, using this as the class labels. Read more in the User Guide.

Code Examples

```
>>> import numpy as np
>>> from sklearn.model_selection import train_test_split
>>> X, y = np.arange(10).reshape((5, 2)), range(5)
>>> X
array([[0, 1],
       [2, 3],
       [4, 5],
       [6, 7],
       [8, 9]])
>>> list(y)
[0, 1, 2, 3, 4]
```

```
>>> X_train, X_test, y_train, y_test = train_test_split(
...     X, y, test_size=0.33, random_state=42)
...
>>> X_train
array([[4, 5],
       [0, 1],
       [6, 7]])
>>> y_train
[2, 0, 3]
>>> X_test
array([[2, 3],
       [8, 9]])
>>> y_test
[1, 4]
```

```
>>> train_test_split(y, shuffle=False)
[[0, 1, 2], [3, 4]]
```

```
>>> from sklearn import datasets
>>> iris = datasets.load_iris(as_frame=True)
>>> X, y = iris['data'], iris['target']
>>> X.head()
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
0                5.1             3.5             1.4
0.2
1                4.9             3.0             1.4
0.2
2                4.7             3.2             1.3
0.2
3                4.6             3.1             1.5
0.2
4                5.0             3.6             1.4
0.2
>>> y.head()
0    0
1    0
2    0
3    0
4    0
...
```

```
>>> X_train, X_test, y_train, y_test = train_test_split(
... X, y, test_size=0.33, random_state=42)
...
>>> X_train.head()
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal
```

```

width (cm)
96          5.7          2.9          4.2
1.3
105         7.6          3.0          6.6
2.1
66          5.6          3.0          4.5
1.5
0           5.1          3.5          1.4
0.2
122         7.7          2.8          6.7
2.0
>>> y_train.head()
96      1
105     2
66      1
0       0
122     2
...
>>> X_test.head()
      sepal length (cm)  sepal width (cm)  petal length (cm)  petal
width (cm)
73          6.1          2.8          4.7
1.2
18          5.7          3.8          1.7
0.3
118         7.7          2.6          6.9
2.3
78          6.0          2.9          4.5
1.5
76          6.8          2.8          4.8
1.4
>>> y_test.head()
73      1
18      0
118     2
78      1
76      1
...

```

Detailed Documentation

Documentation

Split arrays or matrices into random train and test subsets.

Quick utility that wraps input validation, `next(ShuffleSplit().split(X, y))`, and application to input data into a single call for splitting (and optionally subsampling) data into a one-liner.

Read more in the User Guide.

Allowed inputs are lists, numpy arrays, scipy-sparse matrices or pandas dataframes.

If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the test split. If int, represents the absolute number of test samples. If None, the value is set to the complement of the train size. If `train_size` is also None, it will be set to 0.25.

If float, should be between 0.0 and 1.0 and represent the proportion of the dataset to include in the train split. If int, represents the absolute number of train samples. If None, the value is automatically set to the complement of the test size.

Controls the shuffling applied to the data before applying the split. Pass an int for reproducible output across multiple function calls. See Glossary.

Whether or not to shuffle the data before splitting. If `shuffle=False` then `stratify` must be None.

If not None, data is split in a stratified fashion, using this as the class labels. Read more in the User Guide.

List containing train-test split of inputs.

Added in version 0.16: If the input is sparse, the output will be a `scipy.sparse.csr_matrix`. Else, output type is the same as the input type.

Image denoising using kernel PCA

Faces recognition example using eigenfaces and SVMs

Model Complexity Influence

Lagged features for time series forecasting

Probability calibration of classifiers

Probability Calibration curves

Comparison of Calibration of Classifiers

Plot classification probability

Classifier comparison

Recognizing hand-written digits

Column Transformer with Mixed Types

Effect of transforming the targets in regression model

Principal Component Regression vs Partial Least Squares Regression

Multi-class AdaBoosted Decision Trees

Feature transformations with ensembles of trees

Feature importances with a forest of trees

Early stopping in Gradient Boosting

Gradient Boosting Out-of-Bag estimates

Prediction Intervals for Gradient Boosting Regression

Gradient Boosting regression

Gradient Boosting regularization

Features in Histogram Gradient Boosting Trees

IsolationForest example

Comparing random forests and the multi-output meta estimator

Univariate Feature Selection

Examples of Using FrozenEstimator

Failure of Machine Learning to infer causal effects

Common pitfalls in the interpretation of coefficients of linear models

Permutation Importance vs Random Forest Feature Importance (MDI)

Permutation Importance with Multicollinear or Correlated Features

Scalable learning with polynomial kernel approximation

L1-based models for Sparse Signals

Non-negative least squares

Ordinary Least Squares and Ridge Regression

Poisson regression and non-normal loss

Early stopping of Stochastic Gradient Descent

Multiclass sparse logistic regression on 20newgroups

MNIST classification using multinomial logistic + L1

Tweedie regression on insurance claims

Visualizations with Display Objects

Evaluation of outlier detection estimators

ROC Curve with Visualization API

Introducing the set_output API

Post-tuning the decision threshold for cost-sensitive learning

Detection error tradeoff (DET) curve

Custom refit strategy of a grid search with cross-validation

Class Likelihood Ratios to measure classification performance

Multiclass Receiver Operating Characteristic (ROC)

Effect of model regularization on training and test error

Multilabel classification using a classifier chain

Nearest Neighbors Classification

Comparing Nearest Neighbors with and without Neighborhood Components Analysis

Dimensionality Reduction with Neighborhood Components Analysis

Varying regularization in Multi-layer Perceptron

Visualization of MLP weights on MNIST

Restricted Boltzmann Machine features for digit classification

Feature discretization

Map data to a normal distribution

Importance of Feature Scaling

Target Encoder's Internal Cross fitting

Release Highlights for scikit-learn 0.22

Release Highlights for scikit-learn 0.23

Release Highlights for scikit-learn 0.24

Release Highlights for scikit-learn 1.4

Release Highlights for scikit-learn 1.5

Release Highlights for scikit-learn 1.7

Semi-supervised Classification on a Text Dataset

Post pruning decision trees with cost complexity pruning

Understanding the decision tree structure

© Copyright 2007 - 2025, scikit-learn developers (BSD License).